Subscription Plan, Tenant Management & Runtime Feature Access

Requirements & Implementation Document (Complete)

1. Overview

This document defines the full SaaS subscription architecture covering:

Subscription Plan Management

Tenant Management

Tenant–Plan feature mapping

Authentication & authorization

Runtime feature access with session rehydration

Frontend state management using Context API

The solution is designed with senior-level SaaS best practices, ensuring:

Security

Scalability

Maintainability

Clean separation of concerns

2. Scope

In Scope

Subscription plan CRUD

Feature flag management

Tenant creation with plan assignment

Login on tenant domain

Secure runtime feature access

Page refresh resilience

Frontend feature guards

Out of Scope

Payment gateway integration

Billing & invoices

Usage metering

3. User Roles

Role	Access
Super Admin	Manage plans & tenants

Tenant Admin Access tenant features

Tenant User Feature-limited access

4. Subscription Plan Management

4.1 Functional Requirements

Create, edit, activate/deactivate subscription plans

Define limits and feature flags at plan level

Use plan as single source of truth

4.2 Subscription Plan Data Model


```
{
  "id": "uuid",
  "name": "Business",
  "pricing": {
    "monthly": 400,
    "yearly": 4000
  },
  "limits": {
    "maxUsers": 20,
    "maxProjects": 25,
    "maxStorageGb": 100
  },
  "features": {
    "advancedReporting": true,
    "apiAccess": true,
    "prioritySupport": true,
```

```
    "sso": true
  },
  "status": "ACTIVE"
}
```


5. Backend APIs – Subscription Plans

API	Purpose
GET /api/subscription-plans	Listing
GET /api/subscription-plans/{id}	Plan details
POST /api/subscription-plans	Create
PUT /api/subscription-plans/{id}	Update
GET /api/subscription-plans/active	Tenant dropdown

6. Tenant Management

6.1 Tenant Creation Requirements

Tenant selects one subscription plan

Limits are auto-derived

No feature customization at tenant level

6.2 Tenant Data Model


```
{
  "id": "uuid",
  "tenantName": "Acme Corp",
  "domain": "acme",
  "subscriptionPlanId": "plan-uuid",
  "status": "ACTIVE",
```

```
  "isolated": false
}
```

7. Tenant–Subscription Mapping (Core Design)

Key Principle

Tenants never store features.

Tenant → references subscriptionPlanId

Plan → defines limits & features

Feature updates propagate instantly

8. Backend APIs – Tenant Management

API	Purpose
POST /api/tenants	Create tenant
GET /api/tenants/{id}	Tenant with resolved plan
GET /api/tenants/by-domain	Resolve tenant from domain

9. Authentication Architecture

9.1 Token Strategy (Critical)

Token	Storage	Purpose
Access Token (JWT)	Memory only	Authentication
Refresh Token	HttpOnly Cookie	Session persistence

Why two tokens?

Access token expires quickly → secure

Refresh token renews silently → seamless UX

No sensitive data in browser storage

10. Login Flow (Tenant Domain)

1. User visits https://acme.app.com

2. User logs in

3. Backend returns:

  - accessToken (JWT)

  - refreshToken (HttpOnly cookie)

4. Frontend stores accessToken in memory

5. Frontend immediately calls /auth/session

11. Option 1 — Session Rehydration (Core Extension)

11.1 Why Session Rehydration Is Required

Problem:

Context & memory state is lost on refresh

Solution:

Always re-fetch session + features from backend

12. Session Rehydration API

12.1 API Contract

GET /api/auth/session

Authorization: Bearer <accessToken>

12.2 Backend Responsibilities

Validate access token

Resolve tenant

Resolve subscription plan

Return features & limits

12.3 Response


```
{
  "user": {
    "id": "user-uuid",
    "role": "TENANT_ADMIN"
  },
  "tenant": {
    "id": "tenant-uuid",
    "domain": "acme"
  },
  "plan": {
    "name": "Business",
    "limits": {
      "maxUsers": 20,
      "maxProjects": 25
    },
```

```
    "features": {
      "apiAccess": true,
      "advancedReporting": true,
      "sso": true
    }
  }
}
```

13. Frontend State Management (Context API)

13.1 Why Context API

Centralized authorization state

No prop drilling

Easy feature guards

Rehydrated after refresh

14. AuthContext Design

14.1 Context State Shape

```
interface AuthState {
  user: User | null;
  tenant: Tenant | null;
  features: Record<string, boolean>;
  limits: Record<string, number>;
  isAuthenticated: boolean;
  loading: boolean;
```

```
}
```

15. App Bootstrap Flow

```
App Load
└── AuthProvider mounts
    └── call /auth/session
        ├── success → populate context
        └── failure → redirect to login
```

16. Feature Access Helpers

16.1 Feature Guard Hook

```
export const useFeature = (feature: string) => {
  const { features } = useAuth();
  return Boolean(features?.[feature]);
};
```

16.2 Usage

```
const canUseApi = useFeature('apiAccess');

{canUseApi && <ApiSettings />}
```

17. Route Protection

```
<ProtectedRoute feature="advancedReporting">
  <ReportsPage />
</ProtectedRoute>
```

18. Refresh Token Flow

```
Access token expired
```

```
└── 401 from API
    └── POST /auth/refresh
        └── new accessToken
            └── retry original request
```

19. Security Considerations

Concern	Solution
Feature tampering	No localStorage
Token theft	HttpOnly cookies
Feature mismatch	Server-side resolution
Plan changes	Dynamic rehydration

20. Upgrade / Downgrade Plan (Zero Effort)

Update subscriptionPlanId

No frontend changes

Next /auth/session reflects new features

21. Frontend Folder Structure (Final)

```
auth/
  ├── AuthProvider.tsx
  ├── useAuth.ts
  ├── ProtectedRoute.tsx

subscription-plans/
tenant-management/
```

22. Key Architectural Decisions (Summary)

✓ Single source of truth → Subscription Plan

✓ No feature storage in browser

✓ Session rehydration on every reload

✓ Context API for runtime access

✓ SaaS-grade security

23. Final Outcome

This architecture delivers:

🔒 Secure authentication

🔁 Refresh-safe feature access

⚙️ Dynamic plan control

🚀 Enterprise-ready SaaS foundation